SOFTWARE ARCHITECTURE

Project #22 - High-Level Architecture - Team 28 - CODEVERSE

1. Document Control

Field	Value
Project	AI-Powered Upskilling Recommendation and Automation System for Students
Project ID	22
Deliverable	Software Architecture — High-Level Architecture
Team	28 28 - CODEVERSE
Team SRNs	PES2UG24AM184, PES2UG24AM217, PES2UG24AM210, PES2UG24AM214
Version	1.0

2. Architecture Purpose

This document defines the high-level software architecture for Project #22. It translates the functional scope into modular components, interfaces, data stores and deployment units. The architecture follows a layered web-application design so the frontend, backend business logic, database and ML recommendation component can evolve independently.

3. Architecture Drivers

• Modularity and maintainability: each major functionality has a separable module.

• Traceability: architecture components map to functional requirements and Jira stories.

• Security: authentication and authorization are enforced before protected student data is accessed.

• Testability: business logic and ML inference are exposed through testable interfaces.

• Scalability for a mini-project: a modular monolith is preferred over unnecessary microservice complexity.

• ML isolation: recommendation inference is separated from UI logic and business rules.

4. Proposed Architecture Style

Layered modular architecture / modular monolith. The frontend communicates with a REST API backend. The backend contains authentication, profile/skill, recommendation, skill-gap, roadmap and progress modules. PostgreSQL stores application data. The ML model is loaded by a dedicated recommendation service/module. This gives clear module boundaries without the operational overhead of multiple independently deployed microservices.

5. High-Level Architecture

Layer	Component	Responsibility	Interface
Presentation	Web Frontend	Login, profile, skills, recommendations, skill gap, roadmap and progress screens.	HTTPS/HTTP REST calls to backend
API / Controller	REST API	Routes requests, validates input, authenticates user and returns structured responses.	JSON over HTTP
Application / Business	Auth; Profile; Skill; Recommendation; Gap; Roadmap; Progress	Core application rules and orchestration.	Internal service interfaces
ML	Recommendation Engine	Transforms profile/skill data into model features and returns ranked career predictions.	Internal Python interface/API
Data	PostgreSQL	Users, profiles, skills, careers, requirements, resources, roadmaps and progress.	SQL/ORM
Engineering	Tests + CI/CD	Automated tests, quality checks, build/deployment automation.	GitHub/Jenkins

6. Logical Architecture Diagram

STUDENT / BROWSER Web Frontend Login • Profile • Skills • Recommendations • Gap • Roadmap • Progress
REST / JSON ↓
BACKEND API Auth • Profile • Skills • Recommendation • Gap • Roadmap • Progress
■ PostgreSQL   ■ ML Recommendation Engine
POSTGRESQL Users • Skills • Careers • Roadmaps • Progress
ML ENGINE Feature Preparation → Model Inference → Ranked Career Output

Data flow: Browser → REST API → application module → database / ML engine → module → REST API → Browser.

7. Module Responsibilities

Module	Key Responsibilities	Owns
Authentication	Registration, login, logout, session/token validation, password handling.	User identity and access state
Profile	Create/read/update student profile and academic information.	Profile data
Skill Management	Skill CRUD, proficiency validation, normalization.	Student skill data
Recommendation	Feature preparation, ML inference, ranking, explanation/result formatting.	Career recommendation result
Skill Gap	Career selection, required-skill retrieval, skill comparison, missing-skill calculation.	Gap result
Roadmap	Roadmap generation, prerequisite ordering, resource association, regeneration.	Learning roadmap
Progress	Item progress, completion, aggregate progress, performance inputs.	Progress records

8. Core Data Model

Entity	Important Fields	Relationships
User	user_id, email, password_hash, created_at	1:1 StudentProfile; 1:M Skills; 1:M Roadmaps
StudentProfile	profile_id, user_id, degree, year, interests	Belongs to User
Skill	skill_id, name, normalized_name	Referenced by StudentSkill and CareerSkill
StudentSkill	user_id, skill_id, proficiency	M:N User↔Skill
Career	career_id, name, description	1:M CareerSkill; 1:M Roadmap
CareerSkill	career_id, skill_id, required_proficiency	M:N Career↔Skill
Recommendation	recommendation_id, user_id, career_id, score, model_version, created_at	Belongs to User/Career
Roadmap	roadmap_id, user_id, career_id, status, created_at	1:M RoadmapItem
RoadmapItem	item_id, roadmap_id, skill_id/resource_id, order_no, progress, status	Belongs to Roadmap
Resource	resource_id, title, type, URL, skill_id	Referenced by RoadmapItem
Progress	progress_id, item_id, progress, status, updated_at	Belongs to RoadmapItem

9. Key REST Interfaces

Method	Endpoint	Purpose	Auth
POST	/api/auth/register	Create student account	No
POST	/api/auth/login	Authenticate student	No
POST	/api/auth/logout	End authenticated session	Yes
GET	/api/profile	Get current student's profile	Yes
PUT	/api/profile	Update current student's profile	Yes
GET/POST	/api/skills	List/add student skills	Yes
PUT/DELETE	/api/skills/{skillId}	Update/delete student skill	Yes
POST	/api/recommendations	Generate ranked career recommendations	Yes
GET	/api/careers/{id}/gap	Calculate selected-career skill gap	Yes
POST	/api/roadmaps	Generate roadmap	Yes
GET	/api/roadmaps/{id}	Retrieve roadmap	Yes
PUT	/api/roadmaps/items/{id}/progress	Update item progress	Yes
GET	/api/progress/summary	Retrieve progress dashboard data	Yes

10. ML Component Boundary

The ML component receives a validated feature representation derived from the student's profile and normalized skills. It returns career classes/scores or ranked candidates. The backend remains responsible for authorization, input validation, persistence, business rules and presentation formatting. This separation prevents ML code from being coupled to frontend behaviour and makes model replacement possible without redesigning the complete application.

11. Security Architecture

• Passwords must never be stored in plaintext; store a secure password hash.

• Protected API endpoints validate the authenticated user before accessing student-specific data.

• Input validation occurs at the API boundary and again where business rules require it.

• Database queries must use parameterized queries/ORM mechanisms.

• Sensitive values must not be written to normal application logs.

• HTTPS should be used when deployed; local development may use HTTP only in a controlled environment.

• Authorization checks must use the authenticated user's identity rather than a user ID supplied by the client.

12. Deployment View

Unit	Deployment Option	Purpose
Frontend	Static web hosting / project server	Serves browser application.
Backend	Python application server	Runs REST API and business modules.
Database	PostgreSQL server/container	Persistent project data.
ML Model	Packaged model artifact loaded by backend/ML module	Career recommendation inference.
CI/CD	Jenkins + GitHub	Build, test and quality pipeline.
Containerization	Docker	Reproducible backend/database development or deployment.

13. Failure and Recovery

Failure	Expected Handling
Invalid user input	Return validation error; do not write invalid data.
Authentication failure	Return controlled unauthorized response; do not expose sensitive credential details.
Database unavailable	Return controlled service error; log diagnostic details without sensitive data.
ML service/model failure	Return controlled recommendation error/fallback message; preserve existing profile/skills.
Missing roadmap resource	Keep roadmap step usable and flag unavailable resource for correction.
Partial progress update failure	Do not report success unless persistence succeeds; retry/re-submit through a controlled request.

14. Quality Attributes

Attribute	Architectural Measure
Maintainability	Modules have clear responsibilities and stable interfaces; avoid business logic in UI.
Testability	Business functions are callable independently; REST endpoints can be integration-tested.
Security	Authentication, authorization, validation and secure password storage are mandatory.
Reliability	Controlled error handling prevents one failed dependency from crashing the user workflow.
Performance	Recommendation and roadmap generation should be measured in the test environment; avoid unnecessary database calls.
Scalability	Stateless API design allows multiple backend instances later; PostgreSQL remains the shared persistent store.
Portability	Docker/run instructions reduce environment-specific differences.

15. Requirement / Jira Traceability

Functional Group	Architecture Components	Jira Stories
Authentication	Frontend + Auth API + User store	US-01–04
Profile	Profile UI + Profile API + StudentProfile	US-03–04
Skills	Skill UI + Skill module + Skill tables	US-05–08
Recommendation	Recommendation API + ML Engine + Recommendation table	US-09–12
Skill Gap	Gap module + Career/CareerSkill/StudentSkill	US-13–16
Roadmap	Roadmap module + RoadmapItem + Resource	US-17–20
Progress	Progress module + Progress tables + Dashboard	US-21–24
Integration/Quality	API integration + tests + CI/CD + deployment	US-25–28

16. Technology Stack

Area	Proposed Technology
Frontend	Web frontend framework selected by team; REST/JSON client
Backend	Python + FastAPI (proposed)
Database	PostgreSQL
ML	Python, Pandas/NumPy as required, scikit-learn model/inference stack
Testing	pytest + manual browser testing
Version Control	Git + GitHub
CI/CD	Jenkins
Containerization	Docker
Planning / Bug Tracking	Jira

17. Design Decisions / Scope Control

• Use a modular monolith for the mini-project instead of microservices; this keeps deployment and debugging manageable.

• Use a dedicated ML module boundary so the recommendation model can be retrained/replaced independently.

• Use a relational database because the project has structured entities and clear relationships.

• Keep external job-portal automation and social-platform integrations outside the current implementation scope to reduce dependency risk.

• Keep the recommendation workflow explainable at the application level by showing contributing skills/gaps where the selected model permits it.

18. Team Ownership

SRN	Primary Functionality	Supporting Responsibility
PES2UG24AM184	Authentication + Student Profile	API integration and security checks
PES2UG24AM217	Skill Management + Career Recommendation	ML integration and data validation
PES2UG24AM210	Skill Gap Analysis + Learning Roadmap	Data model and recommendation interfaces
PES2UG24AM214	Progress Tracking + Platform Integration	CI/CD and system-testing coordination

19. Architecture Review Checklist

■ Every functional module has an identified owner.

■ All frontend-to-backend interactions use defined API contracts.

■ Authentication is applied to all student-specific endpoints.

■ Database entities support recommendation, gap, roadmap and progress workflows.

- ML inference is isolated from presentation logic.
- Test cases can be mapped to modules and requirements.
- GitHub PRs, Jira stories and test evidence can be linked for traceability.